

PROJETO MULTIDISCIPLINAR SGHSS

ALUNO: ESTHER MARA BALDUÍNO DE PAULO **RU:**

SEMESTRE: 5º

POLO DE APOIO: PAP. CONTAGEM

PROF.: WINSTON SEN LUN FUNG, ME.

INTRODUÇÃO

Neste trabalho, apresento uma proposta de desenvolvimento teórico de um Sistema de Gestão Hospitalar e de Serviços de Saúde (SGHSS) para a instituição VidaPlus. A ideia é criar um sistema que ajude a organizar informações de hospitais, clínicas, laboratórios e equipes de atendimento domiciliar. O sistema serviria para cadastrar pacientes, profissionais, consultas, telemedicina e outras funções importantes.

Tentei aplicar os conteúdos estudados na disciplina de Projetos da melhor forma possível. Utilizei conceitos básicos de Engenharia de Software, como requisitos, modelagem, protótipos e testes. O objetivo é simular como seria participar de um projeto real na área da saúde, mesmo que o trabalho ainda seja simples e introdutório.

Requisitos funcionais e não funcionais

Requisitos funcionais:

Pacientes podem se cadastrar, agendar e cancelar consultas, acessar teleconsulta e visualizar seu histórico clínico.

Profissionais de saúde podem gerenciar agenda, registrar e atualizar prontuários, realizar teleconsultas e emitir receitas digitais.

Administradores podem gerenciar cadastros de pacientes e profissionais, controlar internações e gerar relatórios.

O sistema deve permitir realizar videochamadas seguras para telemedicina e registrar prontuário e prescrições durante a consulta online.

Requisitos não funcionais:

Criptografia de todos os dados sensíveis do sistema.

Controle de acesso baseado em perfis (Paciente, Profissional e Administrador).

Conformidade com a LGPD e registro de logs para auditoria.

Disponibilidade mínima de 99,5% com backups regulares.

Interface responsiva e acessível seguindo padrões W3C/WCAG.

Suporte a múltiplas unidades hospitalares (escalabilidade).

Tempo de resposta rápido em operações críticas, como consultas e prontuário.

Diagrama de caso de uso

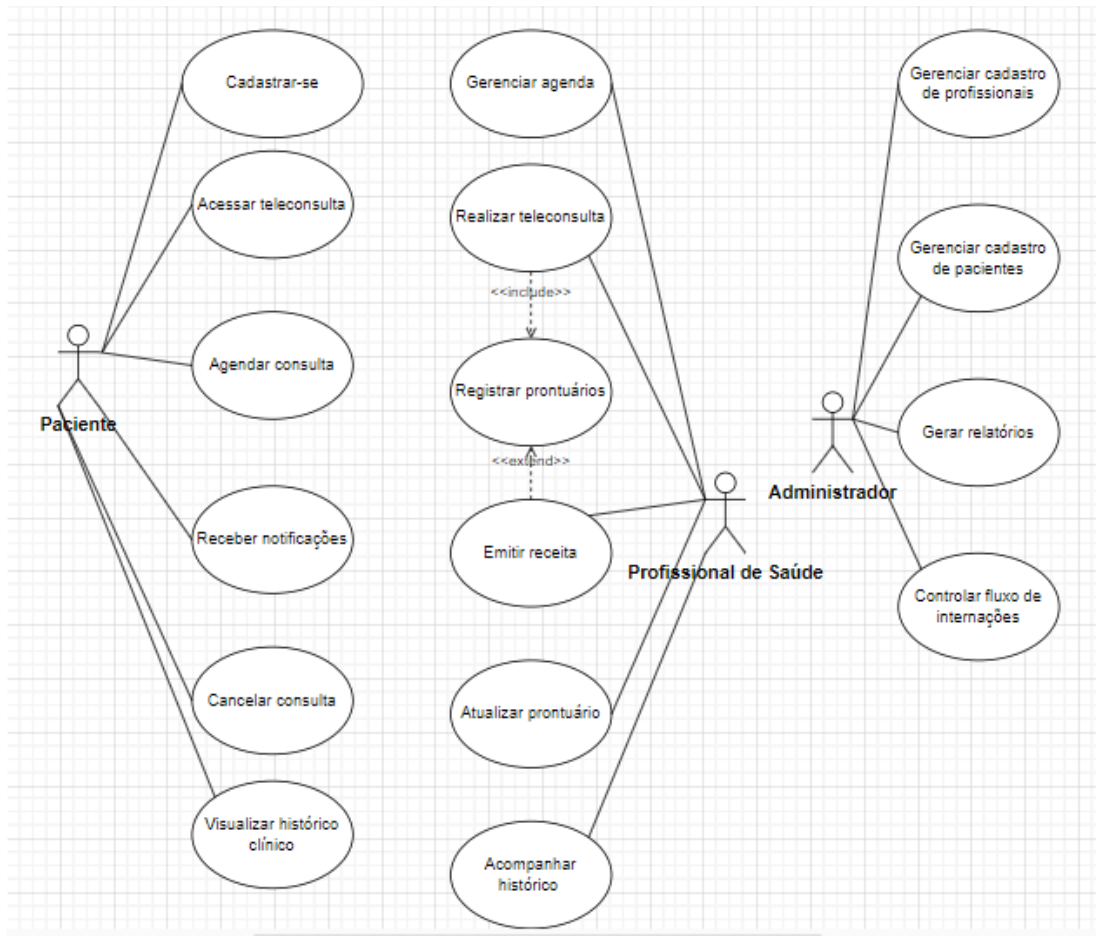


Diagrama de casos de uso com os requisitos funcionais do sistema, representando as ações dos três atores principais: Paciente, Profissional de Saúde e Administrador, incluindo consultas, telemedicina, gestão de prontuários, agenda, cadastros e administração hospitalar

Diagrama de Classes

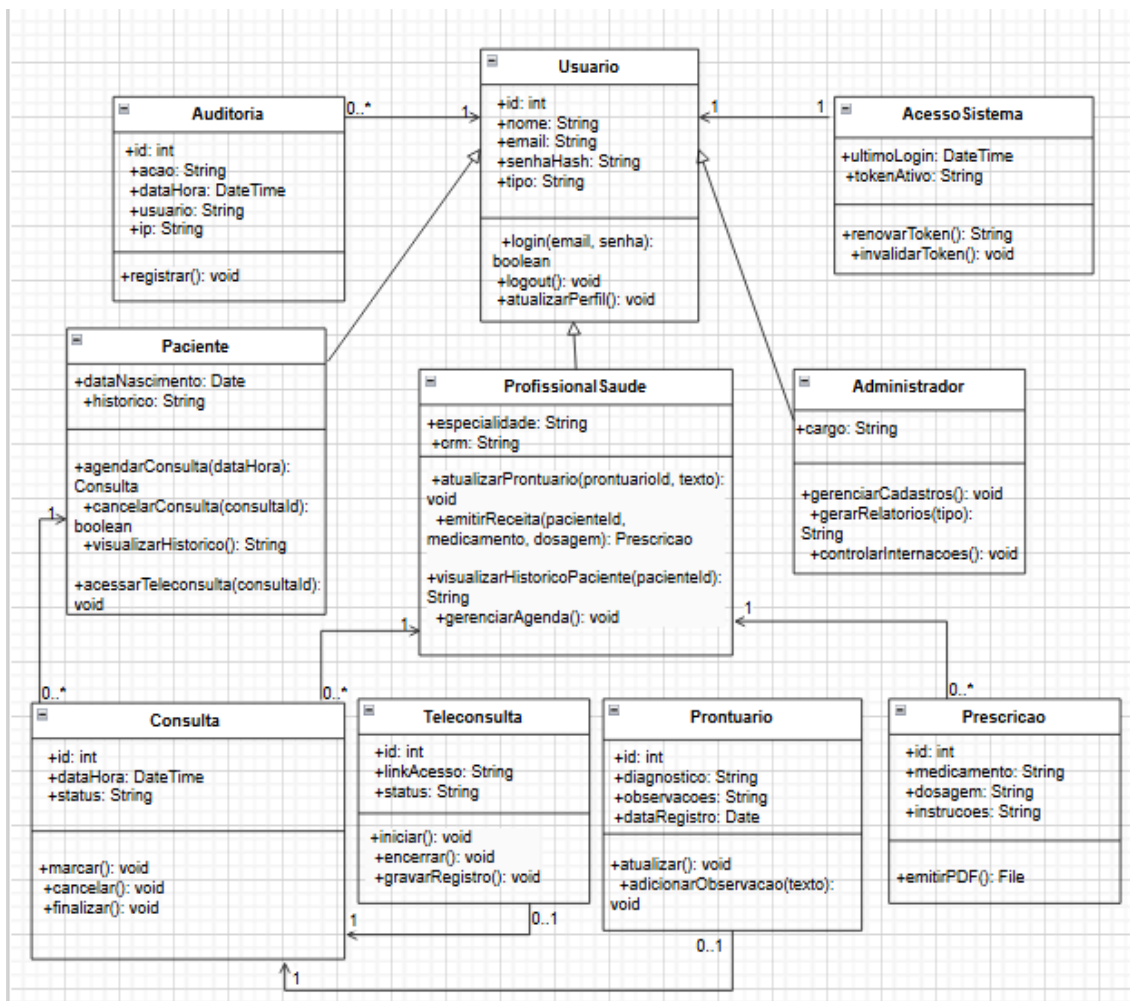


Diagrama de Classes com os principais requisitos funcionais representados: cadastrar usuários e controlar acessos ao sistema; permitir que pacientes agendem, cancelem e realizem teleconsultas; permitir que profissionais de saúde registrem prontuários, emitam prescrições e gerenciem suas agendas; possibilitar que administradores gerenciem cadastros e gerem relatórios; e registrar automaticamente auditorias e eventos de acesso.

Diagrama DER

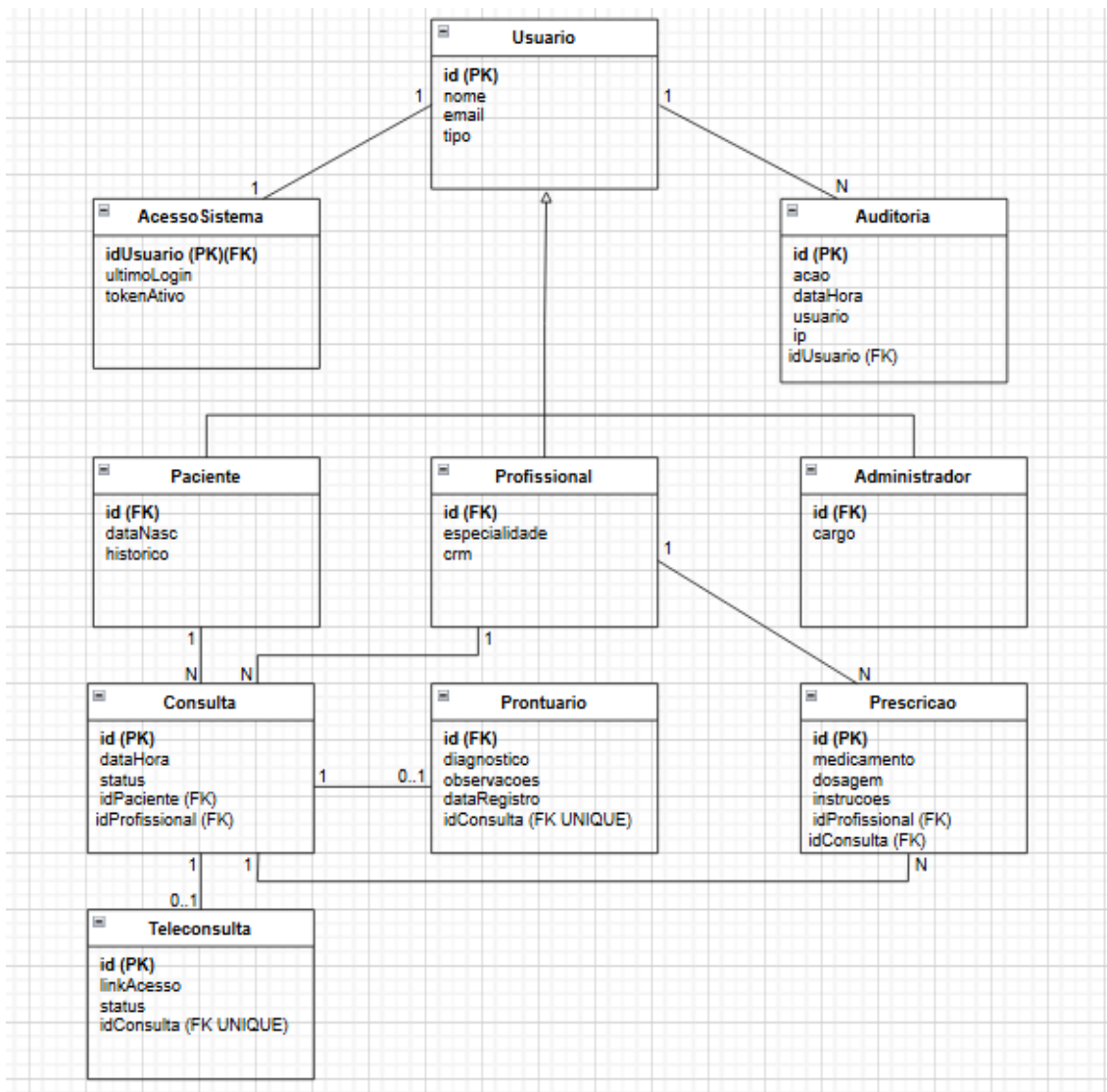


Diagrama Entidade-Relacionamento representando os principais requisitos funcionais do sistema de telemedicina SGHSS: permitir o cadastro de usuários e o controle de acessos ao sistema; garantir que pacientes possam agendar, cancelar e realizar teleconsultas; possibilitar que profissionais de saúde registrem consultas, preencham prontuários, emitam prescrições e acompanhem seus atendimentos; permitir que administradores gerenciem cadastros, perfis e operações do sistema; manter a integridade das atividades clínicas através do registro automático de auditorias e eventos de acesso; registrar informações essenciais sobre consultas presenciais ou remotas, incluindo prontuários, prescrições, teleconsultas, profissionais envolvidos e dados do paciente; e assegurar que todas as operações fiquem vinculadas a um usuário autenticado, garantindo segurança, rastreabilidade e conformidade.

Implementação

Desenvolvida na linguagem C#, Windows Forms (para desktop);

Foram incorporados ao projeto os pacotes NUGET :

Newtonsoft.Json, para comunicação com a API REST;

AForge, AForge.Video, AForge.Video.DirectShow, para manipulação da Webcam;

Foram criadas inicialmente as classes:

Model, chamada CadastroUsuarioModel.cs, para a manipulação dos dados na API;

Controller, chamada apiRotasController.cs, para manipular a comunicação com a API;

Utilizada a IDE Visual Studio Community 2026 para programação da interface;

SOBRE OS TESTES REALIZADOS:

Primeiramente, execução do projeto da API construída em Python;

Logo após, execução do projeto da interface, na IDE de programação;

Escolhida a tela “Cadastro de Usuário” para a realização dos testes (GET, POST, PUT, DELETE);

Escolhida a tela “Teleconsulta” para a demonstração da ativação da Webcam.

Realizando a execução da API.py, no VSCode:

The screenshot shows a Windows IDE with a Python project named PROJETO_SGSHS_API. The left sidebar displays the project structure, including files like PROJETO_SGSHS_API, CONECTOR, funcaoConectar.py, ENDPOINTS, and API.py. The main editor shows the API.py file, which contains imports for Flask and various endpoints, and a Flask app definition. The bottom terminal shows the command to run the application and the output indicating it is running on http://127.0.0.1:5000.

```

1 from flask import Flask
2 from ENDPOINTS.Usuario import usuario_bp
3 from ENDPOINTS.Paciente import paciente_bp
4 from ENDPOINTS.ProfissionalSaude import profissionalSaude_bp
5 from ENDPOINTS.Consulta import consulta_bp
6 from ENDPOINTS.Teleconsulta import teleconsulta_bp
7 from ENDPOINTS.Prontuario import prontuario_bp
8 from ENDPOINTS.Prescricao import prescricao_bp
9 from ENDPOINTS.Auditoria import auditoria_bp
10 from ENDPOINTS.AcessoSistema import acessosistema_bp
11 from ENDPOINTS.Administrador import administrador_bp
12
13 app = Flask(__name__)
14
15 # Registrar os blueprints
16 app.register_blueprint(usuario_bp)
17 app.register_blueprint(paciente_bp)
18 app.register_blueprint(profissionalSaude_bp)
19 app.register_blueprint(consulta_bp)
20 app.register_blueprint(teleconsulta_bp)
21 app.register_blueprint(prontuario_bp)
22 app.register_blueprint(prescricao_bp)
23 app.register_blueprint(auditoria_bp)
24 app.register_blueprint(acessosistema_bp)
25 app.register_blueprint(administrador_bp)
26
27 if __name__ == '__main__':
28     app.run(debug=True)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\m\Documents\GitHub\PROJETO_SGSHS_API> & 'c:\Users\m\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\m\vscode\extensions\ms-python.debugpy-2025.16.0-win32-x64\bundle\libs\debugpy\launcher' '63692' '--' 'c:\Users\m\Documents\GitHub\PROJETO_SGSHS_API\API.py'

* Debug mode: on

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5000

* Press Ctrl+C to quit

* Restarting with stat

* Debugger is active!

* Debugger PIN: 681-514-673

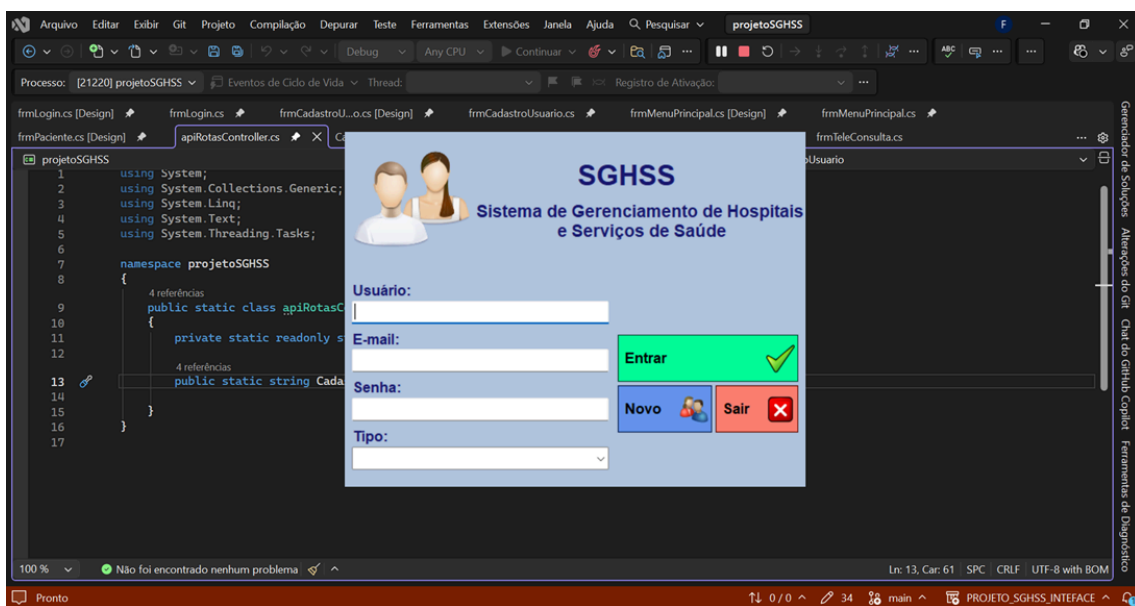
> OUTLINE

> TIMELINE

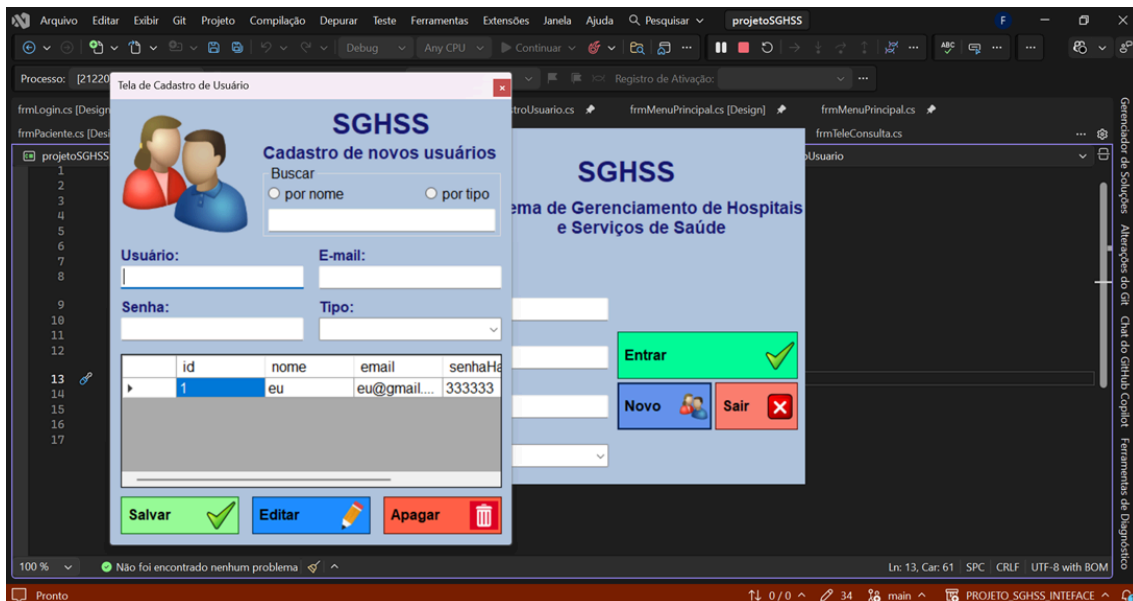
main.py

projetoSGSHS (1 week ago) Ln 17, Col 36 Spaces: 4 UTF-8 LF Python Signed out 3.14.0

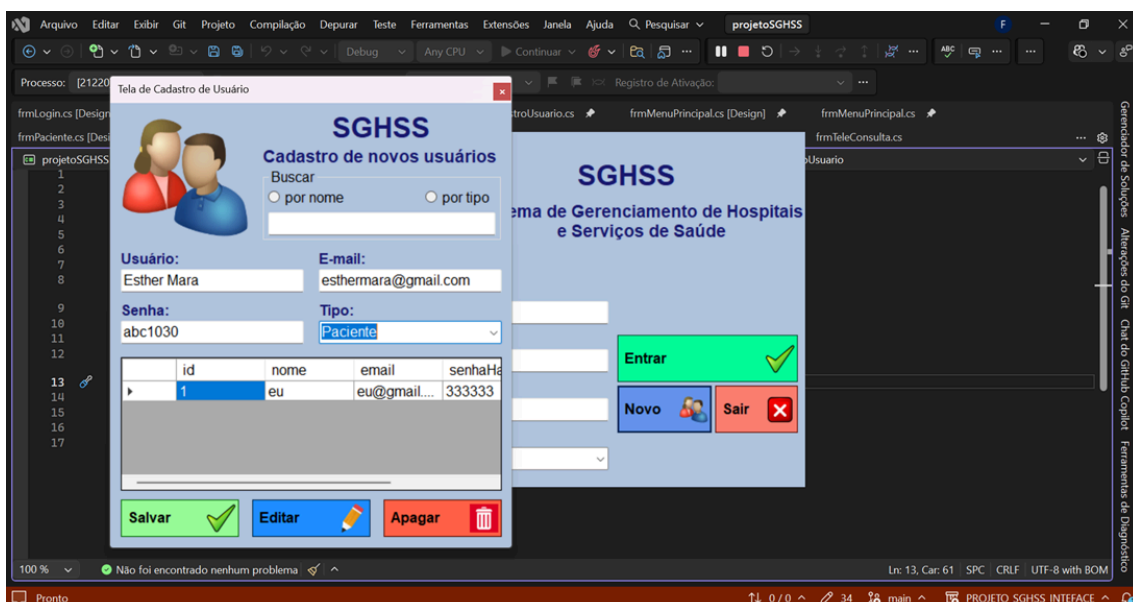
Realizando a execução da interface C#, no Visual Studio Community 2026:



Clicar no botão “Novo”, para novo cadastro de usuário. Nota-se a realização do evento “GET”, trazendo um usuário já cadastrado no banco de dados:

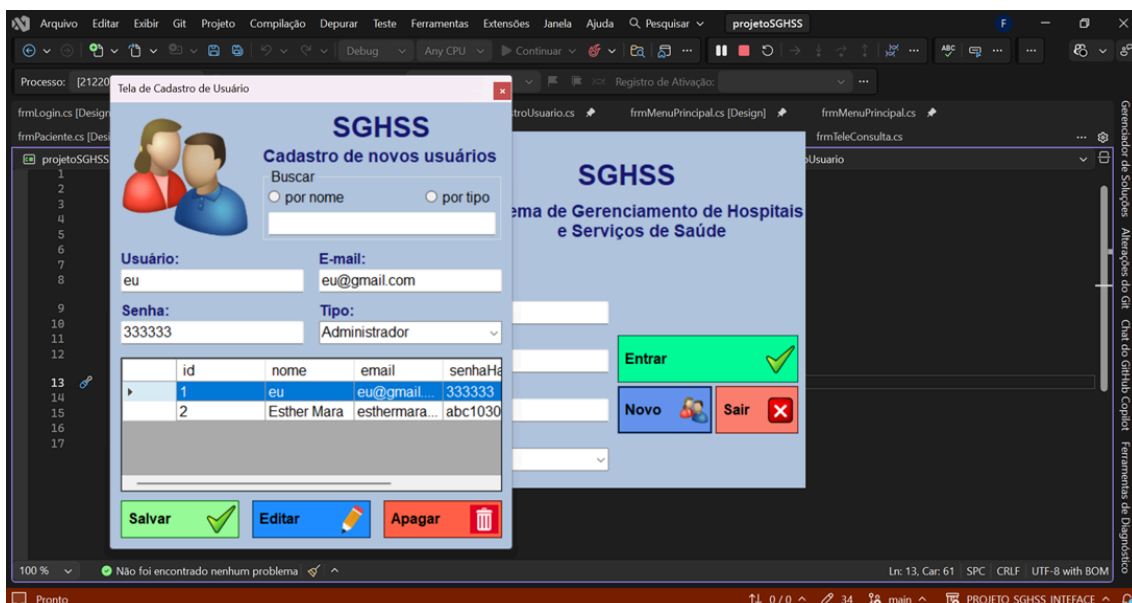


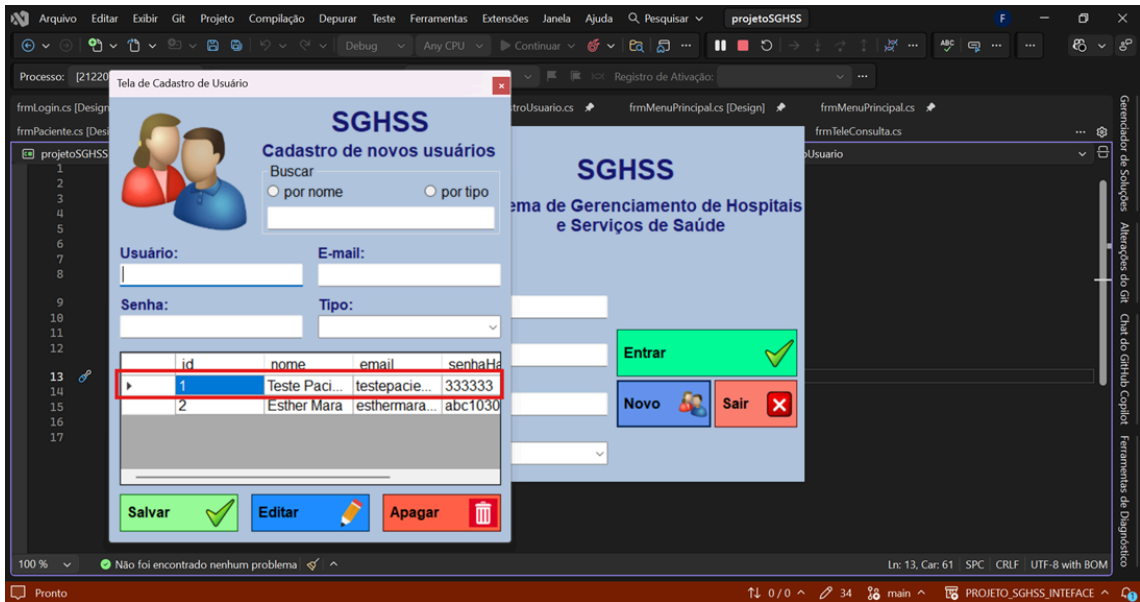
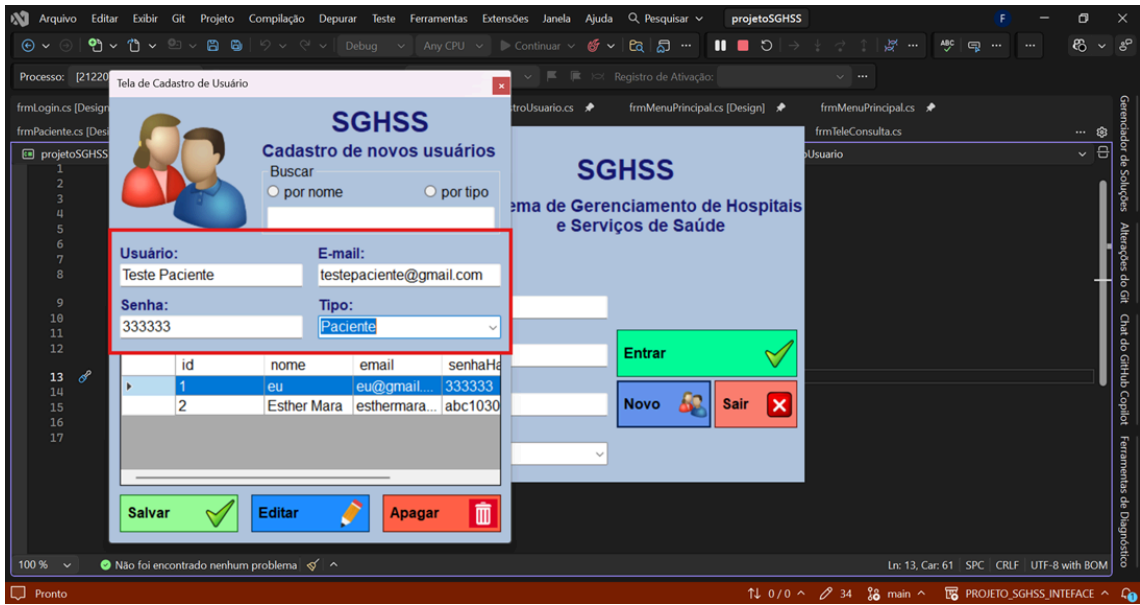
Adicionar: ao preencher os campos com os dados de um novo usuário, clicar no botão “Salvar”. O novo usuário cadastrado aparecerá na grade abaixo:



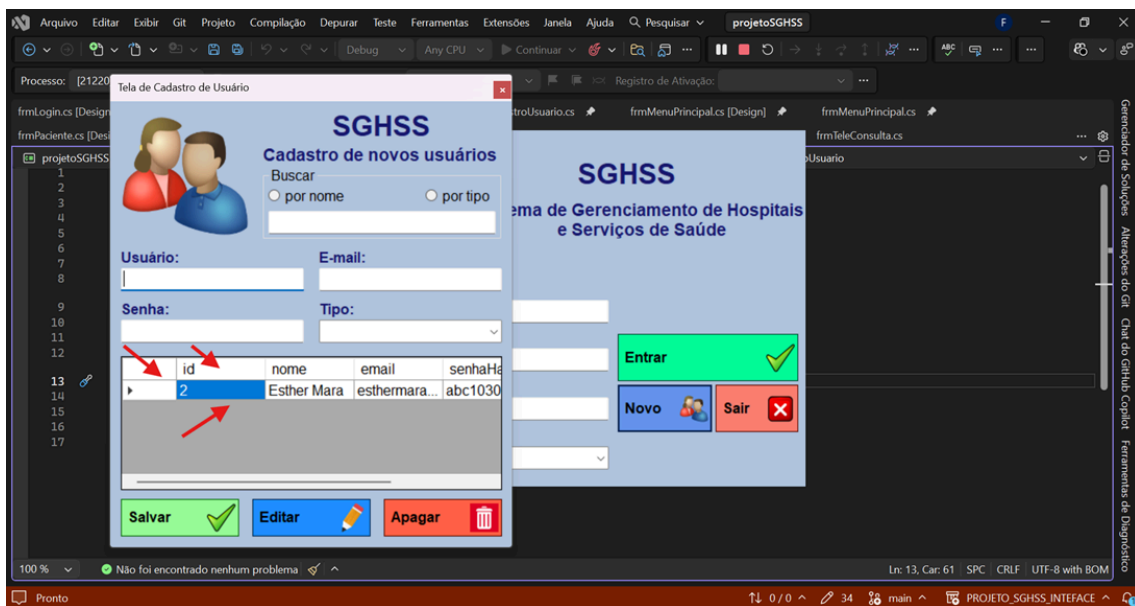
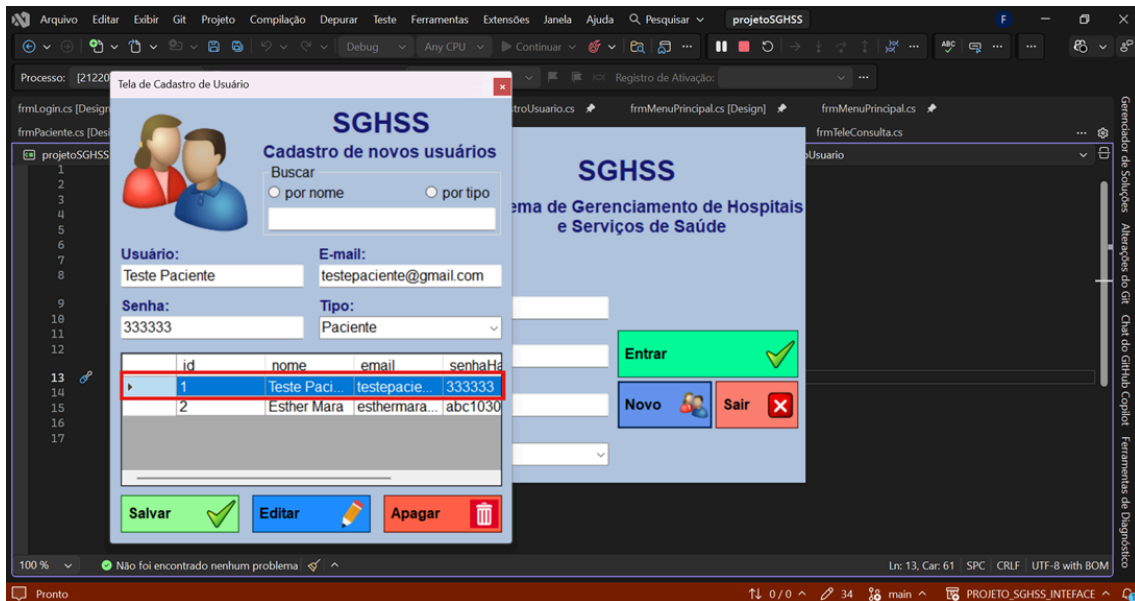


Editar: ao selecionar um usuário cadastrado na tabela abaixo, os dados são populados nos respectivos campos. Realizar uma edição qualquer. Clicar no botão “Editar”. As alterações são salvas:

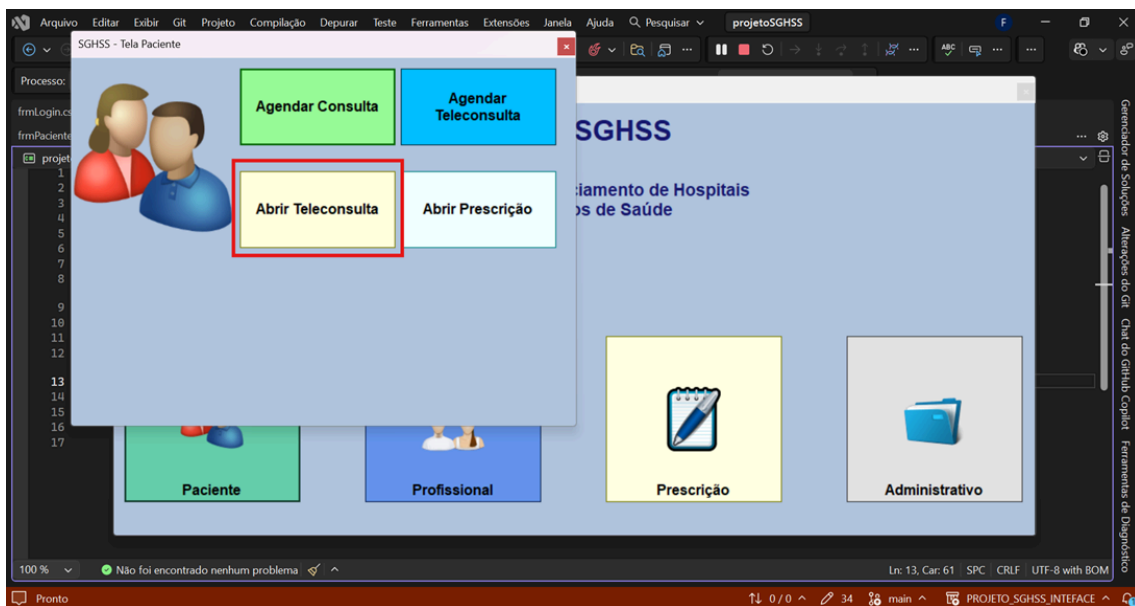




Apagar: ao selecionar um usuário cadastrado na tabela abaixo, os dados são populados nos respectivos campos. Clicar no botão “Apagar”. Os dados são apagados:



Tela Teleconsulta: Clicar no botão “Entrar”. Na tela de “Menu Principal”, clicar no botão “Paciente”. Na tela Paciente, clicar no botão “Abrir Teleconsulta”. Na tela Teleconsulta, clicar no botão “Iniciar Teleconsulta”. A webcam é ativada:



SGHSS - TeleConsulta

Tela do paciente

Iniciar Teleconsulta

Nome do paciente:

tela do profissional

Iniciar Teleconsulta

Nome do profissional:

SGHSS - TeleConsulta



Tela do paciente

Iniciar Teleconsulta

Nome do paciente:

tela do profissional

Iniciar Teleconsulta

Nome do profissional:

API e Teste

Desenvolvida na linguagem Python, com o nome “API.py” e o uso da biblioteca FLASK;

Foi incorporado ao projeto o banco de dados SQLite, chamado “projetoSGHSS.db”;

Os endpoints criados foram:

- AcesoSistema.py
- Administrador.py
- Auditoria.py
- Consulta.py
- Paciente.py
- Prescricao.py
- ProfissionalSaude.py
- Prontuario.py
- Teleconsulta.py
- Usuario.py

Utilizada a IDE VS Code para programação da API e a IDE do SQLite portátil para o banco de dados;

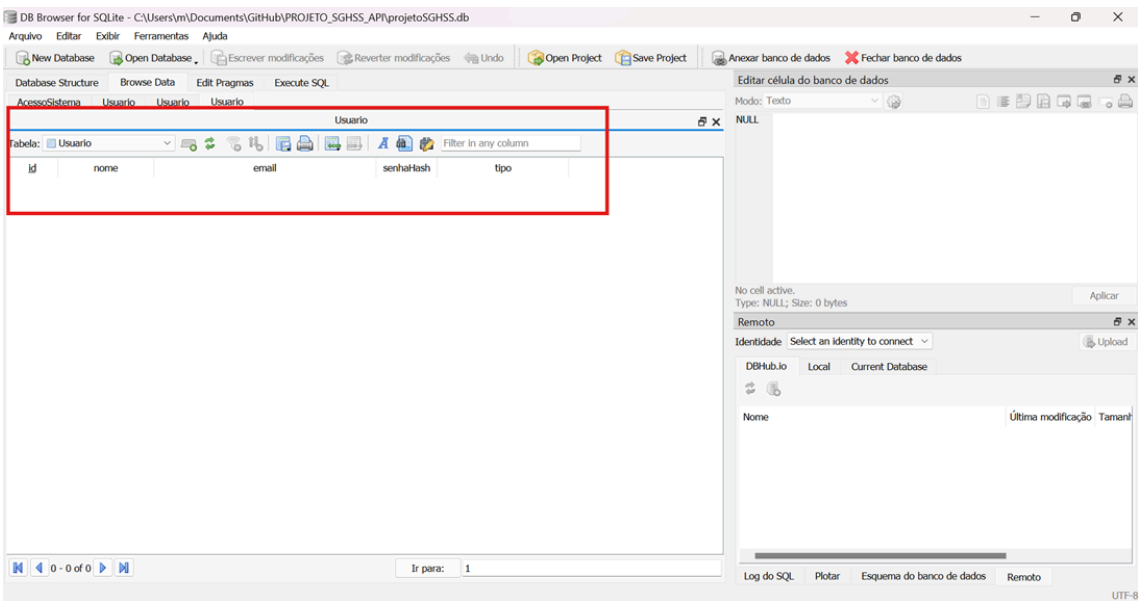
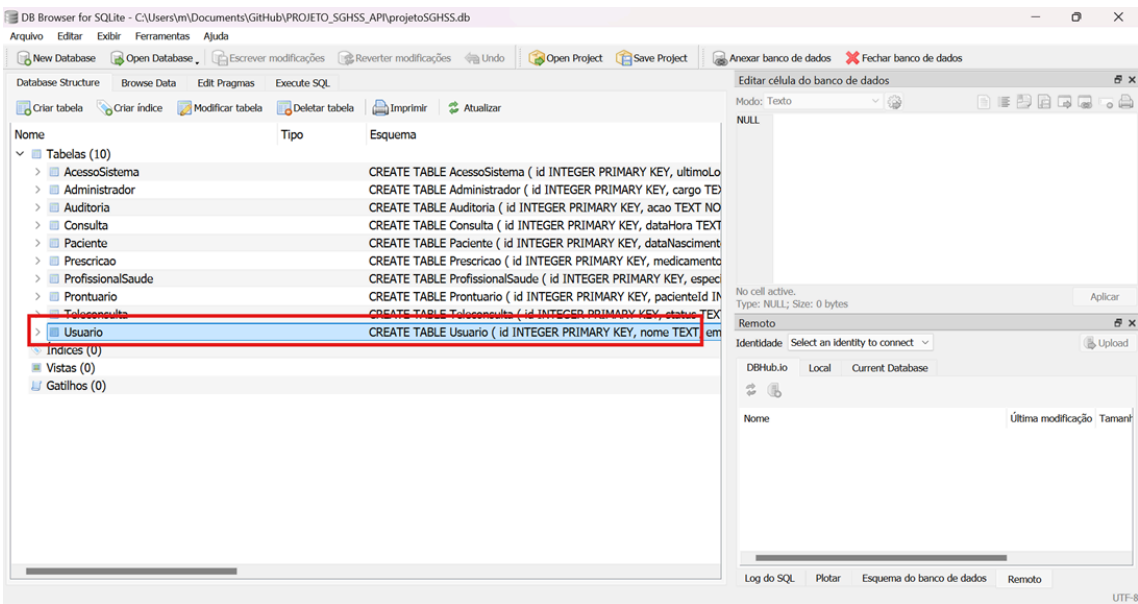
SOBRE OS TESTES REALIZADOS:

Utilizado o programa POSTMAN para a realização dos testes: GET, POST, PUT e DELETE;

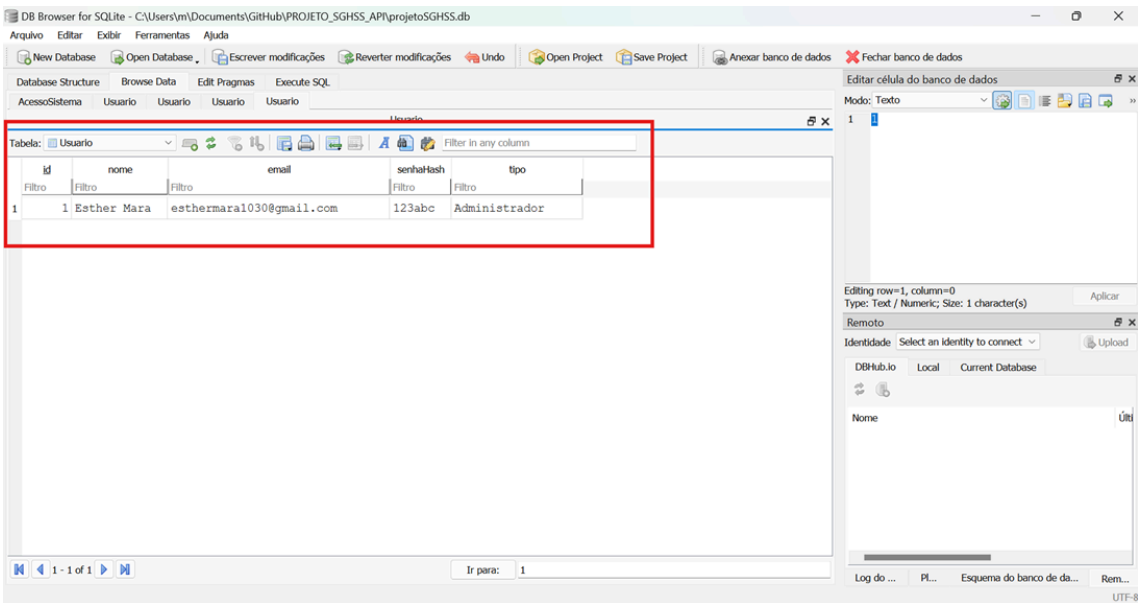
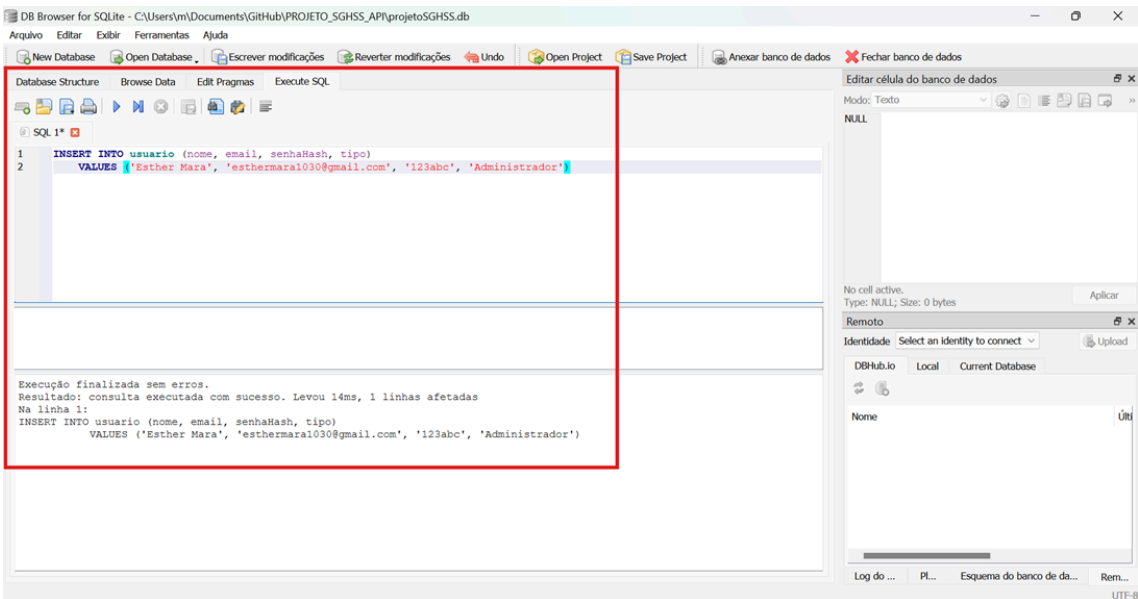
Os resultados foram observados neste programa e confirmados no banco pela IDE do SQLite;

Escolhi o endpoint “Usuario.py” para a realização dos testes.

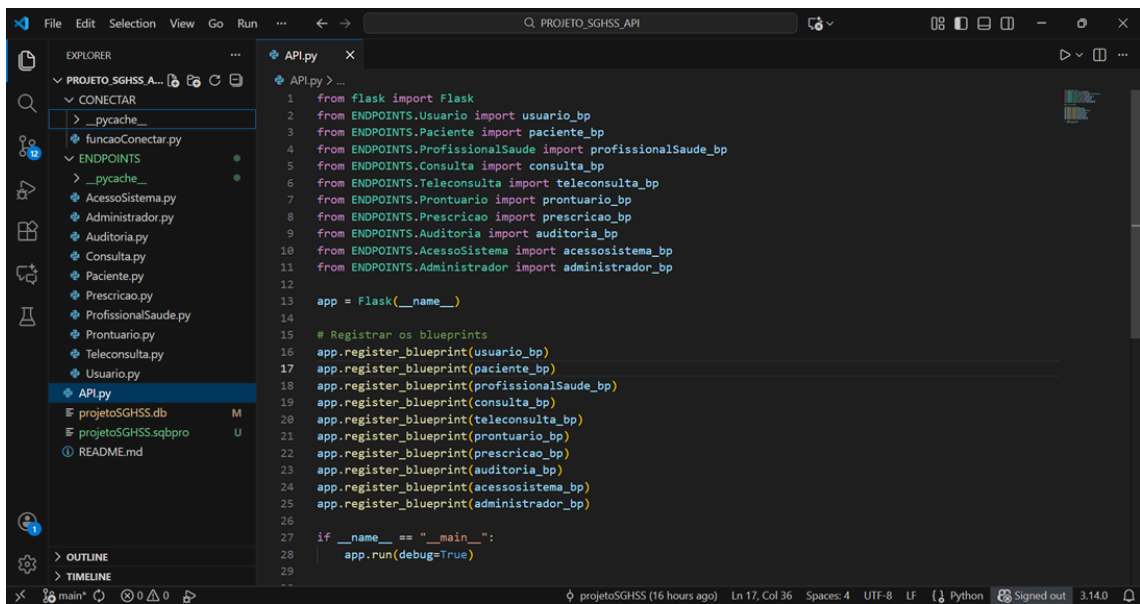
DOCUMENTAÇÃO DOS TESTES REALIZADOS COM O ENDPOINT “USUARIO.PY”:



Realizando um INSERT no banco, por meio de SCRIPT:

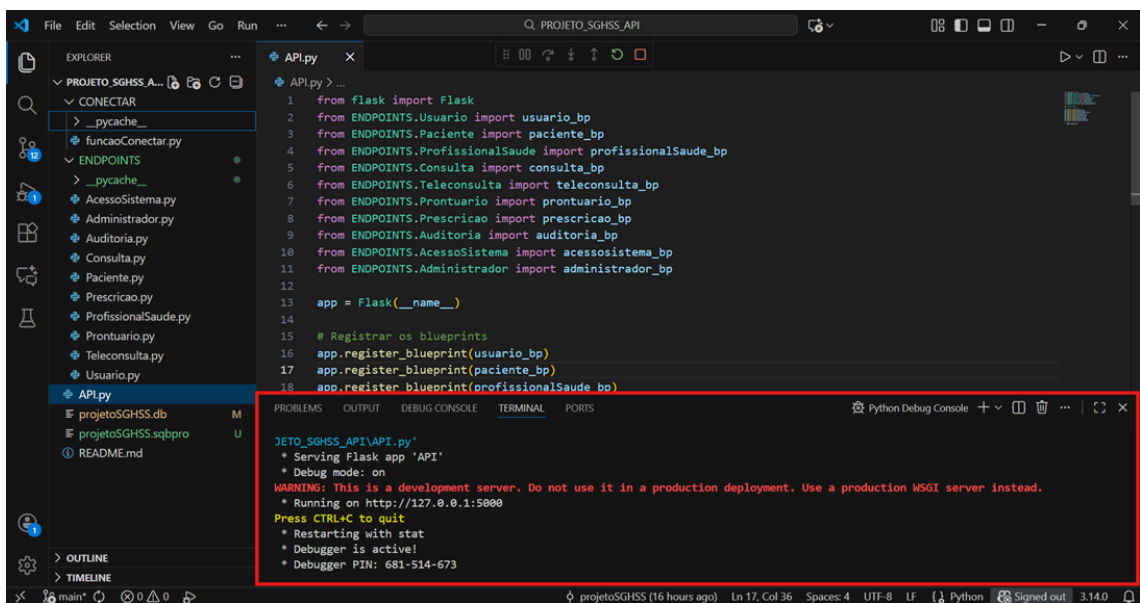


API MONTADA, ENDPOINTS E EM EXECUÇÃO:



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the API.py file open in the editor. The file explorer shows a project structure with folders like PROJECTO_SGHSS_API, CONNECTAR, and ENDPOINTS. The API.py file contains the following code:

```
1 from flask import Flask
2 from ENDPOINTS.Usuario import usuario_bp
3 from ENDPOINTS.Paciente import paciente_bp
4 from ENDPOINTS.ProfissionalSaude import profissionalSaude_bp
5 from ENDPOINTS.Consulta import consulta_bp
6 from ENDPOINTS.Teleconsulta import teleconsulta_bp
7 from ENDPOINTS.Prontuario import prontuario_bp
8 from ENDPOINTS.Prescricao import prescricao_bp
9 from ENDPOINTS.Auditoria import auditoria_bp
10 from ENDPOINTS.AcessoSistema import acessosistema_bp
11 from ENDPOINTS.Administrador import administrador_bp
12
13 app = Flask(__name__)
14
15 # Registrar os blueprints
16 app.register_blueprint(usuario_bp)
17 app.register_blueprint(paciente_bp)
18 app.register_blueprint(profissionalSaude_bp)
19 app.register_blueprint(consulta_bp)
20 app.register_blueprint(teleconsulta_bp)
21 app.register_blueprint(prontuario_bp)
22 app.register_blueprint(prescricao_bp)
23 app.register_blueprint(auditoria_bp)
24 app.register_blueprint(acessosistema_bp)
25 app.register_blueprint(administrador_bp)
26
27 if __name__ == "__main__":
28     app.run(debug=True)
```



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the API.py file open in the editor. The file explorer shows a project structure with folders like PROJECTO_SGHSS_API, CONNECTAR, and ENDPOINTS. The API.py file contains the following code:

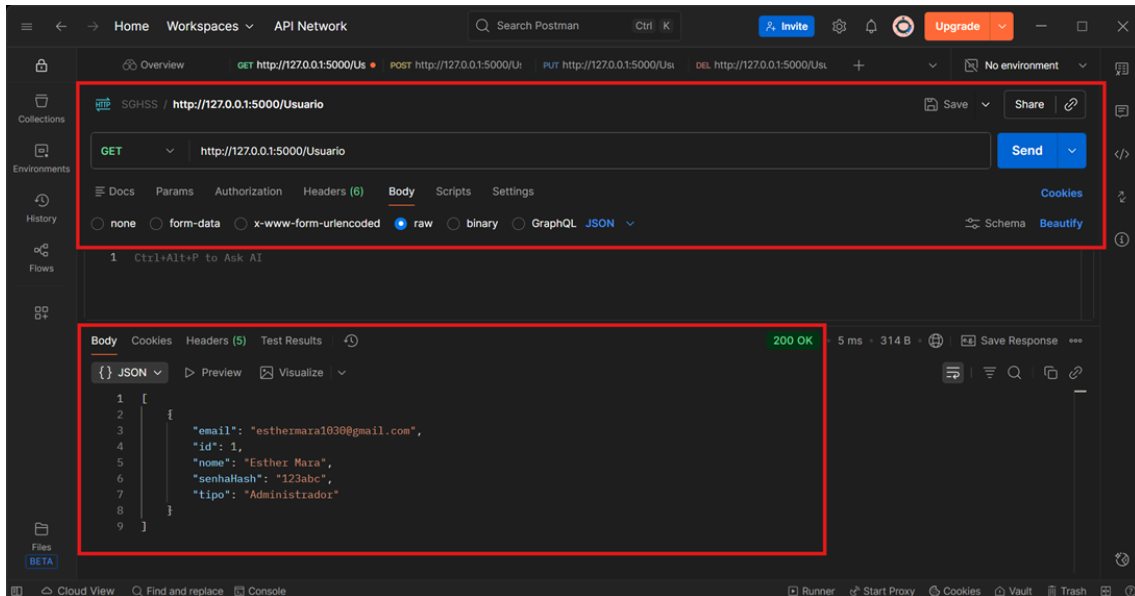
```
1 from flask import Flask
2 from ENDPOINTS.Usuario import usuario_bp
3 from ENDPOINTS.Paciente import paciente_bp
4 from ENDPOINTS.ProfissionalSaude import profissionalSaude_bp
5 from ENDPOINTS.Consulta import consulta_bp
6 from ENDPOINTS.Teleconsulta import teleconsulta_bp
7 from ENDPOINTS.Prontuario import prontuario_bp
8 from ENDPOINTS.Prescricao import prescricao_bp
9 from ENDPOINTS.Auditoria import auditoria_bp
10 from ENDPOINTS.AcessoSistema import acessosistema_bp
11 from ENDPOINTS.Administrador import administrador_bp
12
13 app = Flask(__name__)
14
15 # Registrar os blueprints
16 app.register_blueprint(usuario_bp)
17 app.register_blueprint(paciente_bp)
18 app.register_blueprint(profissionalSaude_bp)
```

The terminal output shows the following messages:

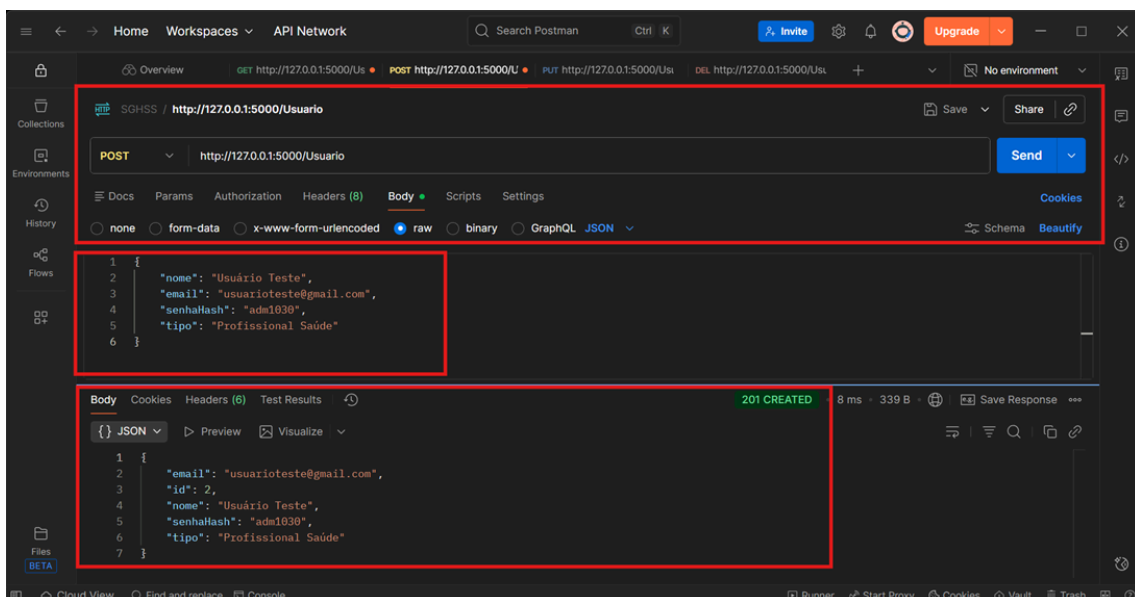
```
JETO_SGHSS_API\API.py'
* Serving Flask app 'API'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 681-514-673
```

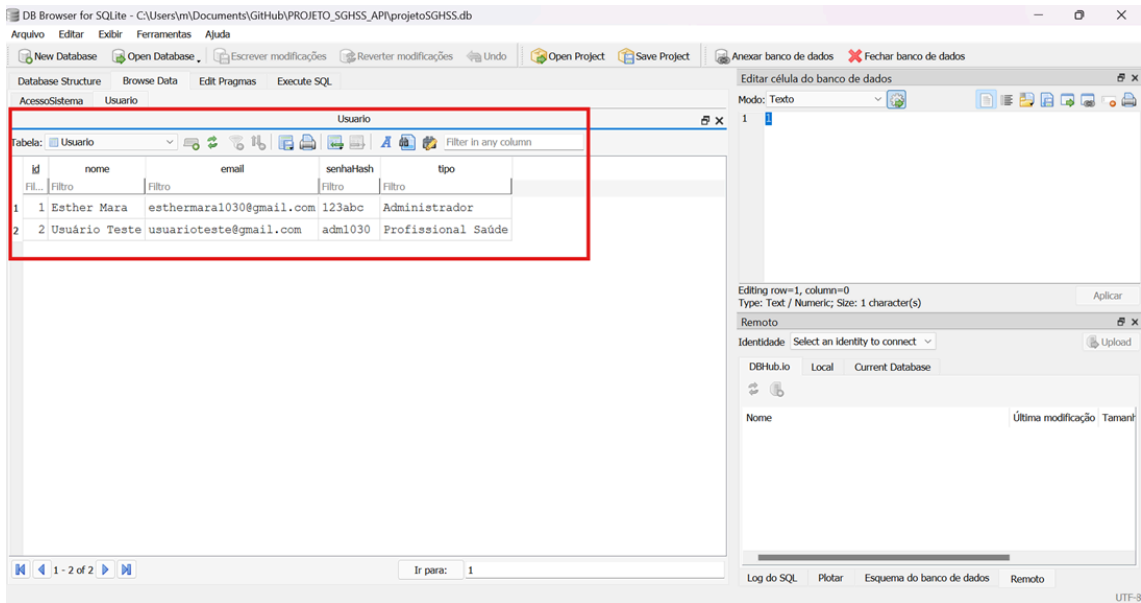
Utilizando o POSTMAN:

Método GET:

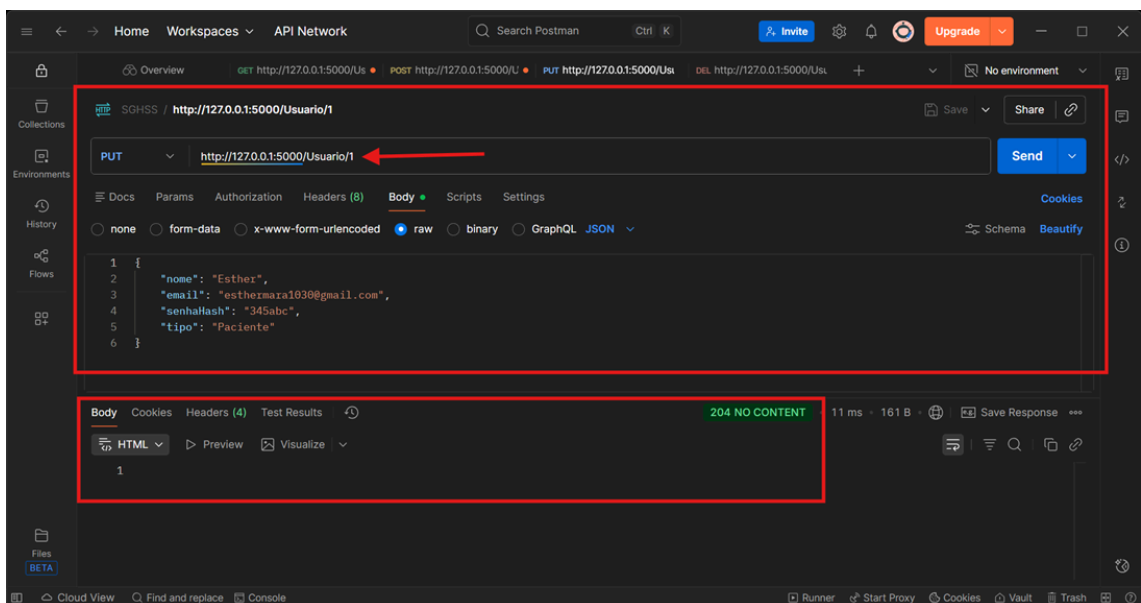


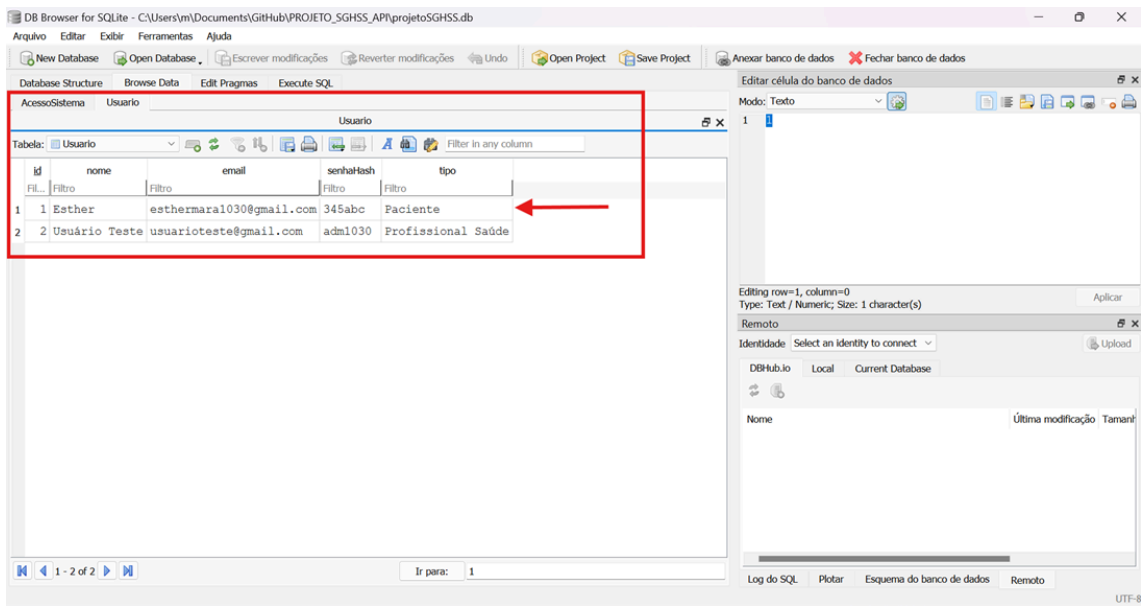
Método POST:



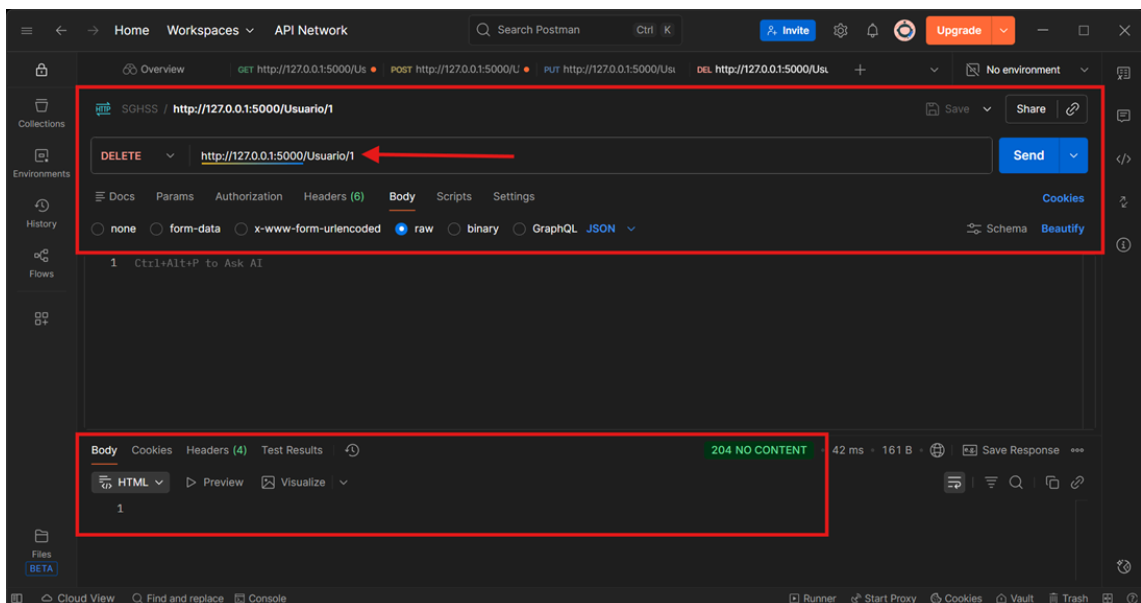


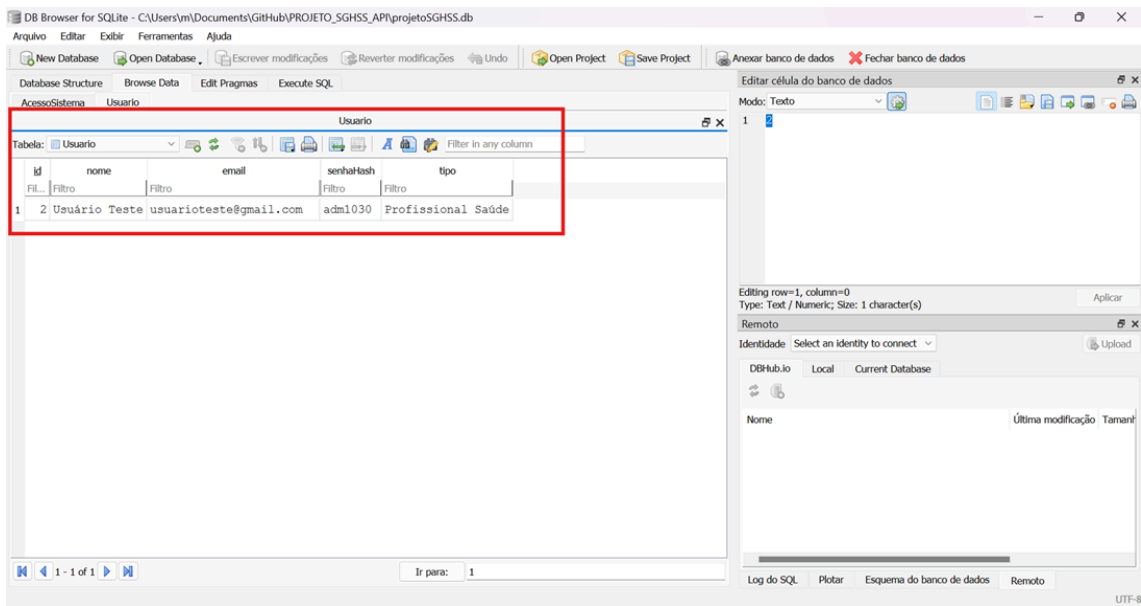
Método PUT:





Método DELETE:





Conclusão

Este projeto me ajudou a entender como funciona o processo de criação de um sistema, desde a fase de requisitos até a prototipação e os testes. Mesmo ainda sendo iniciante, tentei aplicar os conhecimentos da disciplina e montar um modelo simples, mas funcional de um sistema hospitalar. Percebi que criar um sistema dessa área exige bastante atenção, principalmente com segurança e organização das informações. No futuro, pretendo melhorar meus conhecimentos para conseguir implementar um sistema completo e mais avançado.